Learn Tern

Build web pages with a small, strongly typed language

No HTML. No JavaScript. No TypeScript.

A beginner's guide, with exercises and a complete project

Covers Tern 0.3, with structs and libraries. Every example in this guide was run through the real Tern checker.

Contents

Contents 2

1. What is Tern? 4

2. Your first page 5

3. Types and state 6

4. Making pages respond 8

5. Functions 11

6. Decisions and loops 12

7. Building the view 15

8. Lists 17

9. Styling text 19

10. Mixed text, lists and tables 23

11. Named styles 25

12. Pages and navigation 27

13. Buttons and boxes in style 30

14. Fonts 33

15. Images 34

16. Decimal numbers 36

17. Structs: your own types 39

18. Libraries: sharing logic 44

19. Reading error messages 48

20. Project: a quiz 50

21. Exercises 53

Solutions 55

Appendix A: Quick reference 59

1. What is Tern?

Tern is a small language for describing web pages. A Tern page is a plain text file that says what information the page remembers, what it can do, and what it looks like. You write it in the Tern Playground, a page that works on a phone, a tablet or a computer, and the page appears as you type. There is no HTML to write, no JavaScript, and nothing to install.

Tern is **strongly typed**. Every value has a type such as `Int` or `Text`, and Tern checks that you use values correctly before the page ever runs. If you try to add a number to a sentence, you get a clear error message instead of a page that quietly misbehaves.

The three parts of a page

- **state**: the things the page remembers, like a counter or a list of tasks.
- **fn** (functions): the things the page can do, like adding an item.
- **view**: what the page looks like, written in terms of the state.

When you click a button or type in a box, Tern changes the state and then redraws the view. That one idea, **state changes, view redraws**, is the whole mental model.

The shape of every Tern page

```
state count: Int = 0              // remembered

fn bump() { count = count + 1 }   // can change what is remembered

view {                            // always shows the current state
  text "Clicked " + toText(count) + " times"
  button "Click me" -> bump()
}
```

What you need

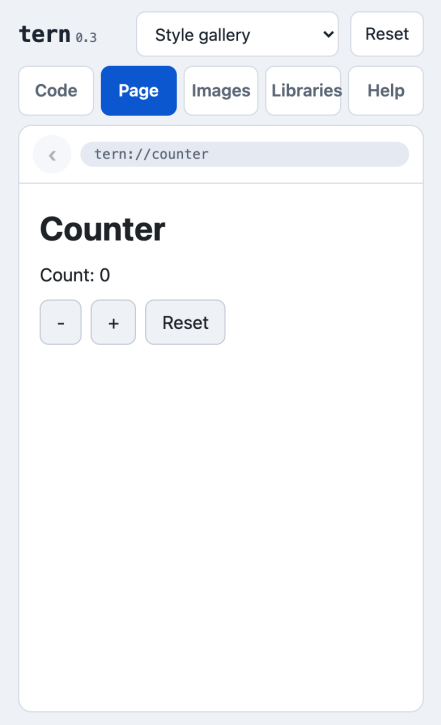
Only the **Tern Playground**. It has four tabs along the top:

- **Code**: where you type. A row of keys above the editor holds the characters that are awkward on a phone keyboard (braces, quote marks, the arrow), and an Insert menu adds ready-made blocks such as a button or a table.
- **Page**: the page you are building. It updates as you type.
- **Images**: sample pictures, and a button to add your own photos (chapter 15).
- **Help**: a cheat sheet you can keep open while you write.

Pick any example from the menu at the top, change it, and watch the page change. **Reset** puts the example back as it was.



The Code tab



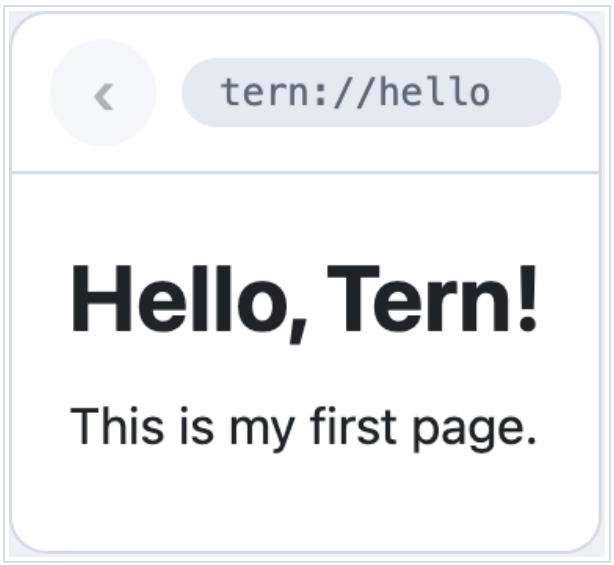
The Page tab

2. Your first page

hello.tern

```
page "Hello"

view {
  heading "Hello, Tern!"
  text "This is my first page."
}
```



The page made by hello.tern

What each line does

- `page "Hello"` gives the page a title (shown in the address bar). It is optional.
- `view { ... }` holds everything the page shows. Every page needs exactly one.
- `heading` shows large text. `text` shows normal text.
- Text goes in double quotes. Inside quotes, `\n` is a new line, `\t` a tab, `\"` a quote mark and `\\` a backslash.
- Anything after `//` on a line is a comment, ignored by Tern.

> **Tip.** There are no semicolons and no commas between lines. Put each statement or element on its own line.

Try it

Open the playground, pick any example, and replace its code with the page above. Change the heading and add a second `text` line, then switch to the Page tab to see it.

3. Types and state

The types

Type	Holds	Examples
`Int`	whole numbers	`0`, `42`, `-7`
`Text`	words and sentences	`"hello"`, `""`
`Bool`	yes or no	`true`, `false`
`List<T>`	an ordered list of values that are all type T	`[1, 2, 3]`, `["a", "b"]`
`Color`	a colour for text or backgrounds (chapter 9)	`red`, `#0b57d0`
`Align`	where text sits: `left`, `center` or `right`	`center`
`Float`	numbers with decimals (chapter 16)	`2.5`, `0.0`
`Font`	a typeface (chapter 14)	`serif`, `mono`

`Int` is for whole numbers, and division of `Int` values rounds down: `7 / 2` is `3`. For decimals use `Float`, which is a different type (chapter 16).

State

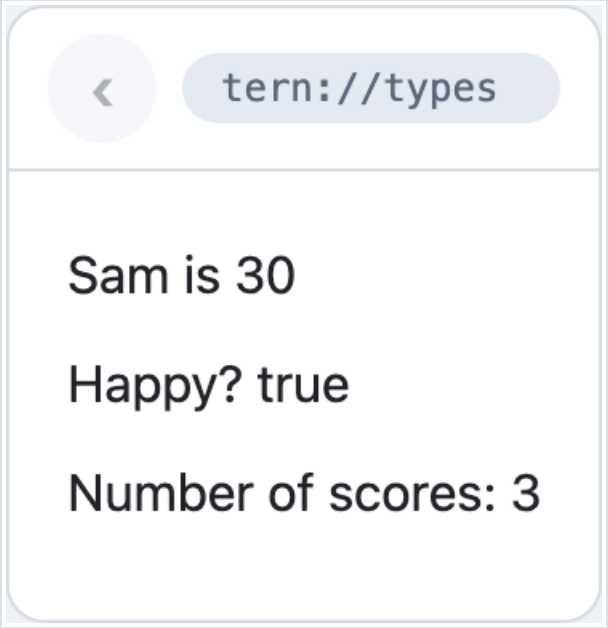
A `state` line declares something the page remembers. It always has a name, a type, and a starting value:

types.tern

```
page "Types"

state name: Text = "Sam"
state age: Int = 30
state happy: Bool = true
state scores: List<Int> = [90, 85, 70]

view {
  text name + " is " + toText(age)
  text "Happy? " + toText(happy)
  text "Number of scores: " + toText(len(scores))
}
```



Four kinds of state, shown with toText()

Watch out. Tern never converts types for you. `"Age: " + age` is an error because `age` is an `Int`. Write `"Age: " + toText(age)` instead. `toText` works on `Int`, `Bool` and `Text`.

4. Making pages respond

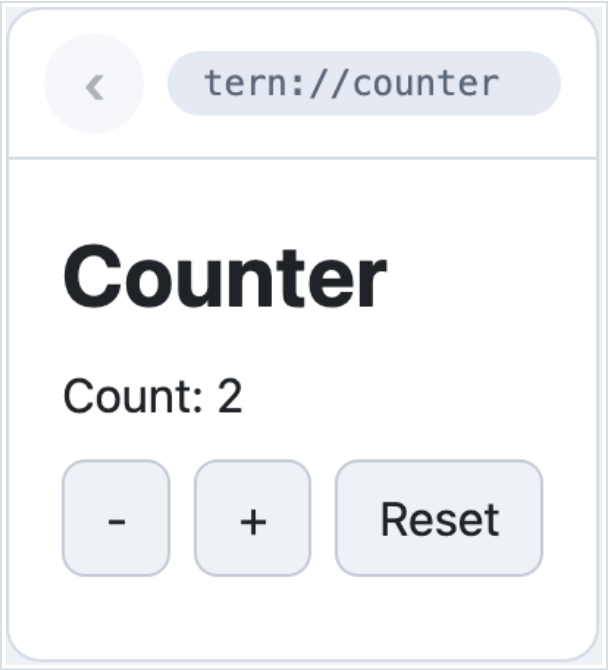
A **button** runs an action when clicked. The action follows an arrow `->`. It can change state, or call a function.

counter.tern

```
page "Counter"

state count: Int = 0

view {
  heading "Counter"
  text "Count: " + toText(count)
  row {
    button "-" -> count = count - 1
    button "+" -> count = count + 1
    button "Reset" -> count = 0
  }
}
```



counter.tern after clicking + three times and - once

`count = count + 1` is an **assignment**: it replaces the value of `count`. Tern checks that the new value has the right type, so `count = "five"` is rejected before the page runs.

Several buttons can sit side by side inside a `row`. Use `column` to stack things vertically (that is also what the page does by default).

Text boxes

An `input` is a text box wired to a `Text` state. Whatever the person types is stored in that state, and the view updates as they type.

```
state name: Text = ""

view {
  input name placeholder "Your name"
  text "Hello, " + name
}
```

> **Note.** The name after `input` must be a state of type `Text`. Tern reports an error if it is missing or is a different type.

`input` can also edit one item of a list, or a `Text` field of a struct in a list: `input names[i]`, `input people[i].name`. It can be a table cell too. The place must start at a state, not at a loop variable (a loop variable is only a copy), so loop over `range(len(xs))` and use the index.

Checkboxes

A `checkbox` shows a `Bool`. Name a `Bool` state and tapping the box flips it: `checkbox agree "I agree"`. To tick one item in a list, give the checkbox any `Bool` and an action: `checkbox picked[i] "Pick" -> picked = setAt(picked, i, !picked[i])`. A checkbox can also be a table cell.

Dropdowns and radio buttons

`select` shows a dropdown and `radio` shows every choice at once. Both pick from a list of `Text` options and store the choice in a `Text` state, or in a place inside one, like `input`: `select country in countries placeholder "Country"` and `radio plan in ["Free", "Pro"]`. If the stored text is not one of the options, nothing is selected. A `select` can also be a table cell: `cells select members[i].role in roles`.

Multi-line text, sliders and disabled controls

`textarea notes placeholder "Notes" height 120` is a multi-line text box on a `Text` state (or a place inside one); `height` and `width` size it. `slider volume in 0 to 100` edits an `Int` between two numbers: a state outside the range is shown at the nearest end, and a range that ends before it starts is a runtime error. Write `disabled` (optionally `disabled when cond`) on a button, text box, textarea, slider, dropdown, radio or checkbox to grey it out. A disabled control does nothing when it is used, for example `button "Send" disabled when !agreed -> sent = true`.

Saved state

Put `saved` in front of a `state` and its value is kept between visits: `saved state laps: List<Int> = []`. States without `saved` start fresh each time. Any type works, including lists and structs. If a kept value no longer fits the type the state now has, it is ignored and the initial value is used. Each page keeps up to 64 KB on this device, only the page itself can read it, and it is never sent anywhere. The playground has a Clear saved button in the page bar; Tern Browser has Clear Saved Data for This Page in the View menu.

Timers

`every 1000 when running -> seconds = seconds + 1` runs the action every 1000 milliseconds while the page is showing and `running` is true. The time is an `Int` in milliseconds. A timer never ticks faster than every 100 ms, a page can have at most four at once, it pauses while the browser tab or the app is in the background, and it stops when you go to another `view`. If its action fails three times in a row, the timer stops and the error is shown.

Dialogs

A `modal` is drawn over the page while its `Bool` is true. The page behind it is dimmed and cannot be tapped. Tapping the dim area, or pressing Escape, closes it: on a plain `Bool` state that sets the state to false, and with `-> action` it runs your action instead. A modal goes directly in the view, not inside a row or column. It takes `width`, `background`, `padding`, `radius` and `border`.

A dialog

```
state confirm: Bool = false

button "Delete" -> confirm = true
modal confirm width 300 {
  heading "Delete it?"
  button "Cancel" -> confirm = false
}
```

5. Functions

A function packages up some work under a name. You list each **parameter** with its type, and after the parentheses you write the type of the value it hands back.

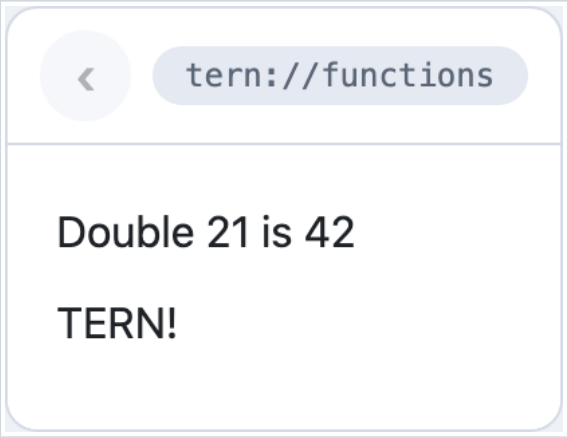
functions.tern

```
page "Functions"

fn double(n: Int): Int {
  return n * 2
}

fn shout(s: Text): Text {
  return upper(s) + "!"
}

view {
  text "Double 21 is " + toText(double(21))
  text shout("tern")
}
```



functions.tern

Rules worth knowing

- `fn name(a: Int, b: Text): Text { ... }` takes two parameters and returns `Text`.
- Leave off `: Type` when a function returns nothing. Such a function is usually called from a button.
- A function that promises a type must `return` a value on **every** path. Tern checks this.
- Parameters and `let` variables are local to the function. `state` is visible everywhere.
- Functions can call each other and themselves (recursion), in any order in the file.
- `let total = 0` creates a local variable. Its type is worked out from the value. Add a type when you need to: `let xs: List<Int> = []`.

> **Tip.** Give small jobs their own function. `button "Add" -> add()` reads much better than five lines of assignments inside the button.

6. Decisions and loops

grades.tern

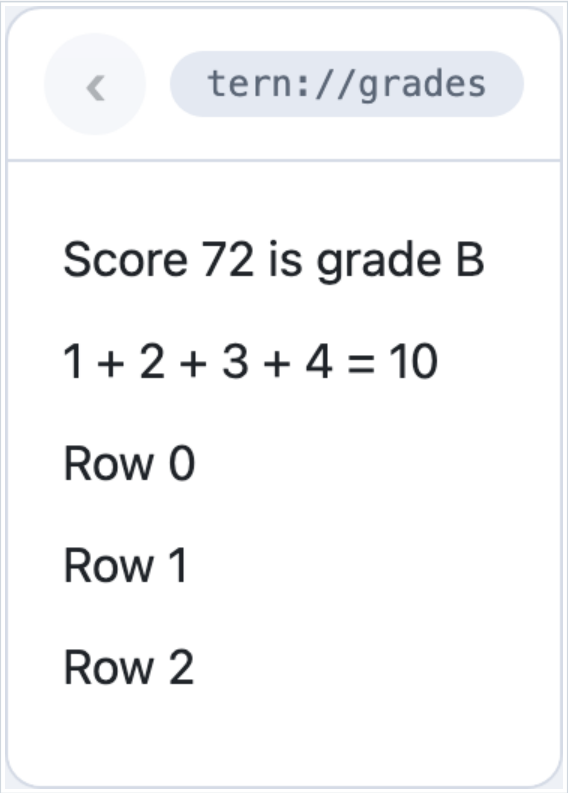
```
page "Grades"

state score: Int = 72

fn grade(s: Int): Text {
  if s >= 90 {
    return "A"
  } else if s >= 70 {
    return "B"
  } else {
    return "C"
  }
}

fn sum(xs: List<Int>): Int {
  let total = 0
  for x in xs {
    total = total + x
  }
  return total
}

view {
  text "Score " + toText(score) + " is grade " + grade(score)
  text "1 + 2 + 3 + 4 = " + toText(sum([1, 2, 3, 4]))
  for n in range(3) {
    text "Row " + toText(n)
  }
}
```



grades.tern

if / else

The condition must be a `Bool`. Writing `if 1 { ... }` is a type error, because 1 is not true or false. Chain choices with `else if`.

Comparing and combining

Kind	Operators
Compare any two values of the same type	`==` `!=`
Compare numbers	`<` `>` `<=` `>=`
Combine yes/no values	`&&` (and) `||` (or) `!` (not)
Arithmetic on `Int`	`+` `-` `*` `/` (rounds down) `%` (remainder)
Join `Text`, or join two lists	`+`

for and foreach loops

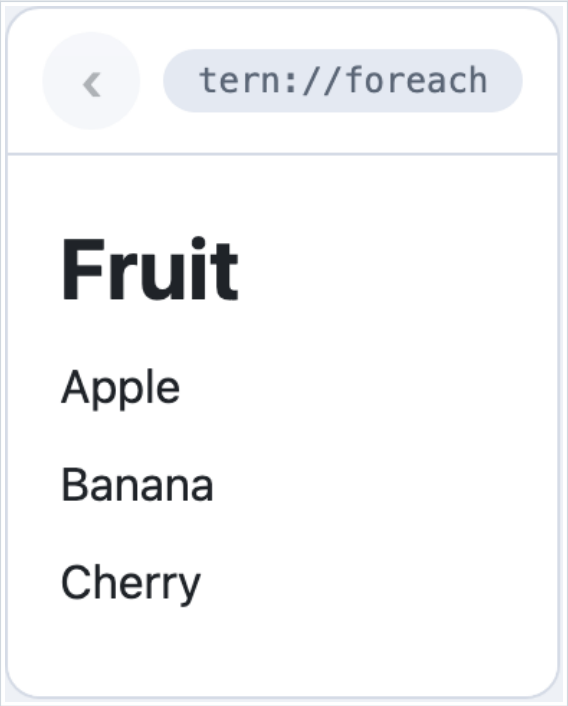
`for x in xs { ... }` (or `foreach x in xs { ... }`) runs the block once per item of a list. `foreach` means the same thing as `for`. To repeat something a number of times, loop over `range(n)`, which is the list `[0, 1, ..., n-1]`. There is no `while` loop, so a Tern page cannot get stuck looping forever.

foreach_demo.tern

```
page "Foreach"

state fruits: List<Text> = ["Apple", "Banana", "Cherry"]

view {
  heading "Fruit"
  foreach fruit in fruits {
    text fruit
  }
}
```



foreach_demo.tern, drawn with foreach

7. Building the view

Element	What it does
`heading expr`	Large text. The expression must be `Text`.
`text expr`	Normal text.
`button expr -> action`	A button. The action is one statement, or a `{ block }`.
`input name placeholder expr`	A text box bound to a `Text` state. The placeholder is optional.
`modal flag { ... }`	A dialog drawn over the page while the `Bool` is true. Tapping the dim area or pressing Escape closes it.
`textarea place`	A multi-line text box on a `Text` state; `height`, `width`, optional `placeholder`.
`slider place in low to high`	A slider that edits an `Int` between low and high.
`saved state name: T = value`	A state whose value is kept between visits (up to 64 KB per page).
`every ms when cond -> action`	Run an action every `ms` milliseconds (at least 100) while the page shows and the condition is true.
`select place in options`	A dropdown that stores the chosen `Text`; optional `placeholder`.
`radio place in options`	Radio buttons that store the chosen `Text`.
`checkbox flag "label"`	A checkbox bound to a `Bool` state; or `checkbox expr "label" -> action`.
`row { ... }` / `column { ... }`	Lay children out side by side, or stacked.
`if cond { ... } else { ... }`	Show different things depending on state.
`for x in list { ... }`	Repeat elements, one per list item. `foreach` works the same way.
`link text to address`	A link to a web address. Use `link text -> goto page` to move inside your own site.
`image expr`	A picture (chapter 15).
`para { span ... }`	A paragraph with mixed styles (chapter 10).
`bullet expr`	A bulleted line.
`table height 240 { ... }`	A table; with height it scrolls in a box, header fixed (chapter 10).
`divider`	A thin horizontal line.

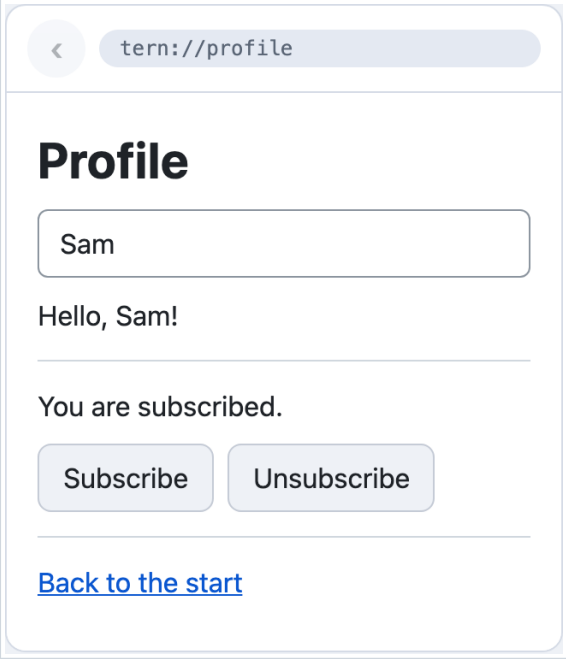
profile.tern

```
page "Profile"

state name: Text = ""
state subscribed: Bool = false

fn status(): Text {
  if subscribed {
    return "You are subscribed."
  }
  return "You are not subscribed."
}

view {
  heading "Profile"
  input name placeholder "Your name"
  if trim(name) != "" {
    text "Hello, " + trim(name) + "!"
  }
  divider
  text status()
  row {
    button "Subscribe" -> subscribed = true
    button "Unsubscribe" -> subscribed = false
  }
  divider
  link "Back to the start" to "hello.tern"
}
```



profile.tern after typing a name and clicking Subscribe

Inside a view, `if`, `for` and `foreach` choose and repeat **elements**. Inside a function or button action, they choose and repeat **statements**. The words are the same, and the contents differ.

8. Lists

A list holds any number of values of one type. Count from zero: the first item is `xs[0]`.

Code	Result
`[10, 20, 30]`	a `List<Int>`
`xs[1]`	the item at position 1 (counting from 0)
`len(xs)`	how many items (also works on `Text`)
`append(xs, 5)`	a **new** list with 5 added at the end
`removeAt(xs, 0)`	a **new** list without the item at position 0
`setAt(xs, 1, 9)`	a **new** list with position 1 replaced by 9
`slice(xs, 1, 3)`	a **new** list of positions 1 and 2 (never fails if the range runs past the end)
`sort(xs)`	a **new** list, smallest first (`Int`, `Float` or `Text` lists)
`sortOrder(xs)`	the positions of `xs` in sorted order, so one list can sort another
`reverse(xs)`	a **new** list, back to front
`range(4)`	`[0, 1, 2, 3]`
`join(["a", "b"], ", ")`	`"a, b"`

Lists never change in place. `append` and `removeAt` return a new list, so you store the result back into your state: `items = append(items, "Bread")`.

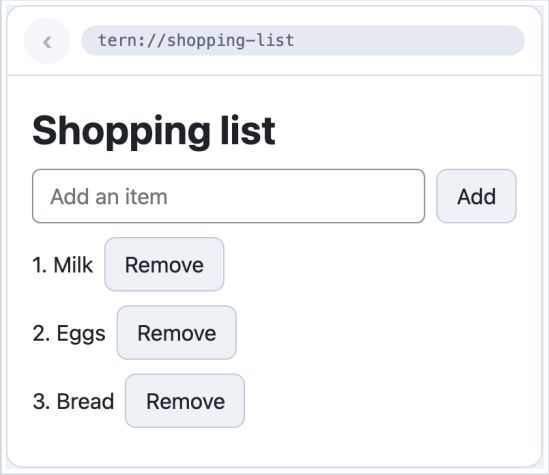
shopping.tern

```
page "Shopping list"

state items: List<Text> = ["Milk", "Eggs"]
state draft: Text = ""

fn add() {
  if trim(draft) != "" {
    items = append(items, trim(draft))
    draft = ""
  }
}

view {
  heading "Shopping list"
  row {
    input draft placeholder "Add an item"
    button "Add" -> add()
  }
  if len(items) == 0 {
    text "Nothing to buy."
  }
  for i in range(len(items)) {
    row {
      text toText(i + 1) + ". " + items[i]
      button "Remove" -> items = removeAt(items, i)
    }
  }
}
```



shopping.tern after adding Bread

Watch out. Index outside a list, as in `xs[5]` when it has 3 items, causes a runtime error. Tern shows it in the status bar and keeps the page running. An empty list `[]` needs a known type: `state xs: List<Int> = []`.

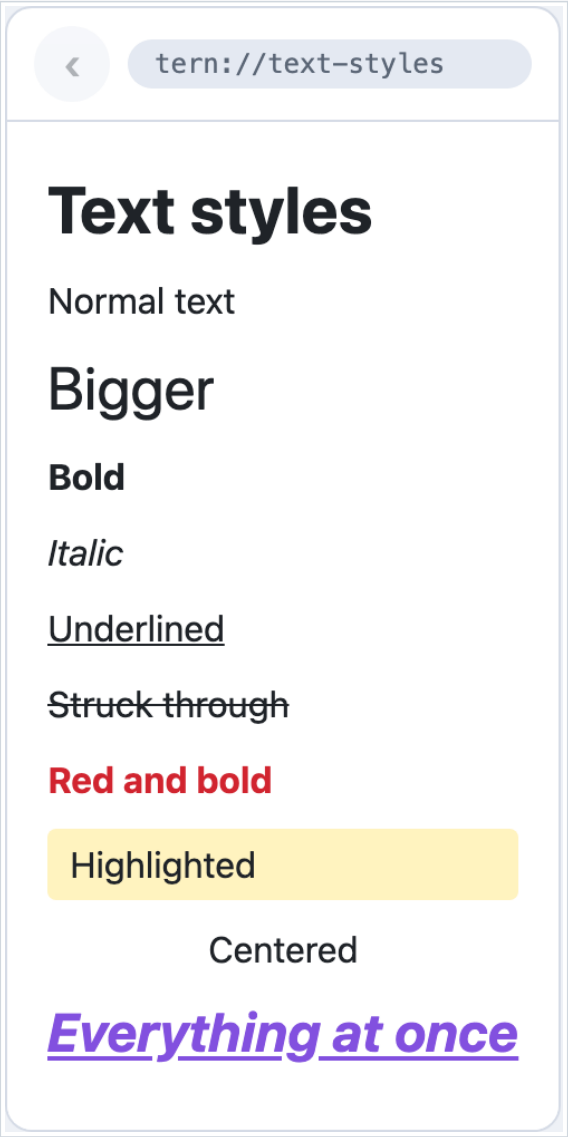
9. Styling text

So far every page has used one plain look. To change how a piece of text looks, add words after it. The order does not matter, and you can add as many as you like.

style_text.tern

```
page "Text styles"

view {
  heading "Text styles"
  text "Normal text"
  text "Bigger" size 20
  text "Bold" bold
  text "Italic" italic
  text "Underlined" underline
  text "Struck through" strike
  text "Red and bold" color red bold
  text "Highlighted" background #fff3bf
  text "Centered" align center
  text "Everything at once" size 18 bold italic underline color purple
}
```



Every kind of text style

Add this	To get
`size 20`	bigger or smaller text, in points. Normal text is 12; a heading is 22.
`bold` `italic` `underline` `strike`	that effect
`color red`	coloured text
`background #fff3bf`	a highlight behind the text
`align center`	`left`, `center` or `right`

Colours

A colour can be a name, a hex code such as `#0b57d0` (or the short form `#fff`), or `rgb(r, g, b)` with each part from 0 to 255. The names are `black white gray red green blue orange purple yellow pink`.

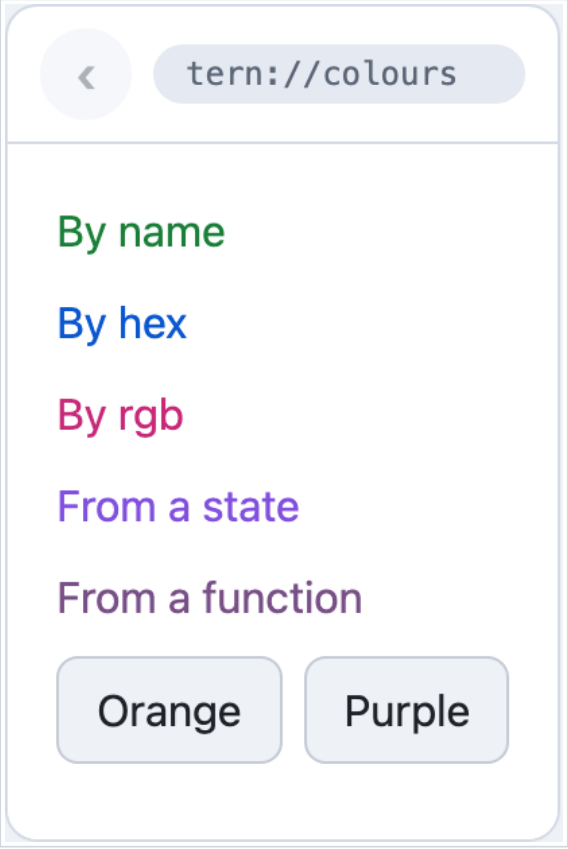
colors.tern

```
page "Colours"

state tint: Color = orange

fn shade(level: Int): Color {
  return rgb(level, 80, 255 - level)
}

view {
  text "By name" color green
  text "By hex" color #0b57d0
  text "By rgb" color rgb(200, 40, 120)
  text "From a state" color tint
  text "From a function" color shade(120)
  row {
    button "Orange" -> tint = orange
    button "Purple" -> tint = purple
  }
}
```



colors.tern after tapping Purple

> **Watch out.** `Color` is a type, like `Int`. The word `red` is a colour, but `"red"` in quotes is `Text`, and Tern will tell you so: `color "red"` is an error.

Styles that change

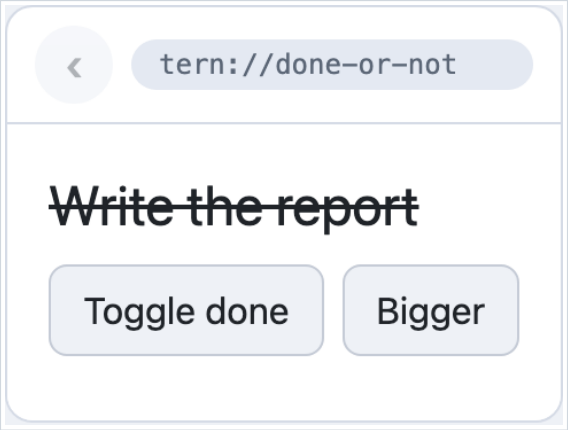
Every number and flag in a style can come from state, so the style follows the page. Put `when` after `bold`, `italic`, `underline` or `strike` to switch it on only while a condition is true. `size` and `color` take any expression of the right type.

dyn.tern

```
page "Done or not"

state done: Bool = false
state pts: Int = 14

view {
  text "Write the report" strike when done bold when !done size pts
  row {
    button "Toggle done" -> done = !done
    button "Bigger" -> pts = pts + 2
  }
}
```



After tapping Toggle done, then Bigger twice

What can be styled

Text, headings and bullets take everything above. A `button` takes size, bold, italic, color and background. A `link` takes size, bold, italic, strike and color. A `row` or `column` takes `background` and `padding`. Asking for something an element cannot have is a type error, for example "a button cannot have 'align'".

10. Mixed text, lists and tables

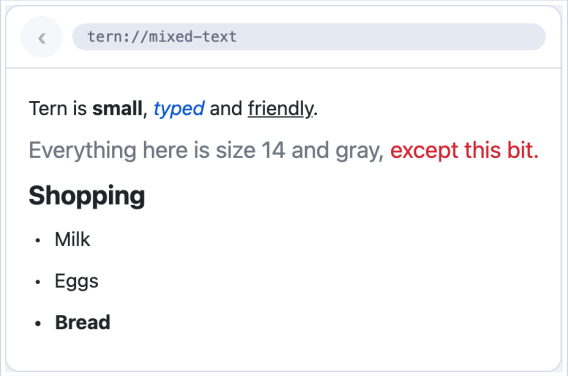
Mixing styles in one sentence

A `para` holds `span` pieces that flow together as one paragraph. Settings on the `para` are shared by every span, and a span can change them for itself. A `bullet` is a single bulleted line.

para.tern

```
page "Mixed text"

view {
  para {
    span "Tern is "
    span "small" bold
    span ", "
    span "typed" italic color blue
    span " and "
    span "friendly" underline
    span "."
  }
  para size 14 color gray {
    span "Everything here is size 14 and gray, "
    span "except this bit." color red
  }
  heading "Shopping" size 16
  bullet "Milk"
  bullet "Eggs"
  bullet "Bread" bold
}
```



A mixed paragraph and a bullet list

Tables

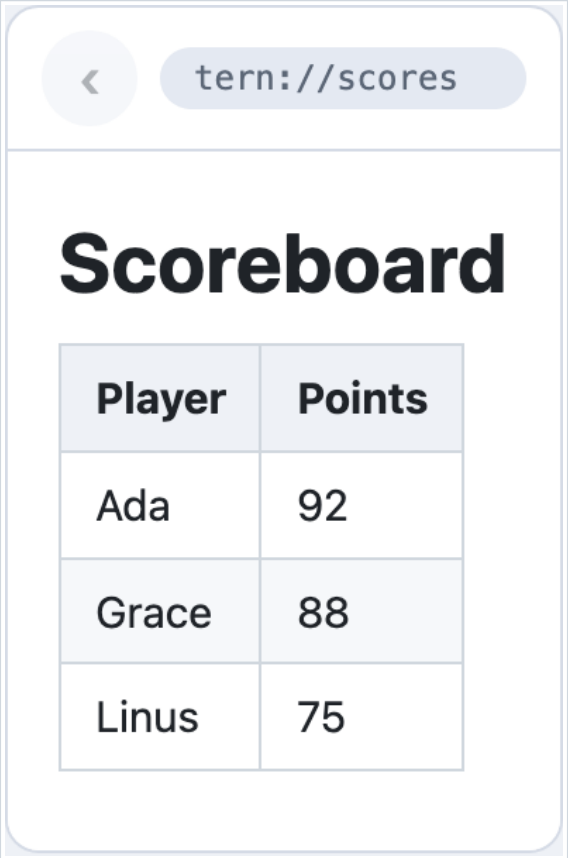
A `table` has an optional `header` line and any number of `cells` lines. Separate the cells with commas. Use `for` and `if` inside a table to build rows from your lists.

tables.tern

```
page "Scores"

state names: List<Text> = ["Ada", "Grace", "Linus"]
state points: List<Int> = [92, 88, 75]

view {
  heading "Scoreboard"
  table {
    header "Player", "Points"
    for i in range(len(names)) {
      cells names[i], toText(points[i])
    }
  }
}
```



A table filled from two lists

- Cells are `Text`. Use `toText()` for numbers.
- The `header`, if there is one, comes first and appears once.
- Every row must have the same number of cells. Tern checks this before the page runs.
- A table wider than the screen scrolls sideways. For a long table, write `table height 240 { ... }`: it then scrolls up and down in its own box (240 points tall) and the header row stays in view.

- A cell can be a `button` or a `link` (so a header can sort, and a first column can pick rows), and `span 2` makes a cell cover two columns. Style words after a cell, like `bold` or `background #eef`, apply to that cell.
- The table itself takes `padding`, `size`, `color`, `background`, `border` and `font`.
- Write `table resizable { ... }` and the reader can drag the right edge of a header cell to change that column's width. The widths last until the page is reloaded.

A row with too few cells

```
view {
  table {
    header "Name", "Qty"
    cells "Tea"
  }
}
```



Tern says: this row has 1 cell(s) but the table has 2 column(s)

11. Named styles

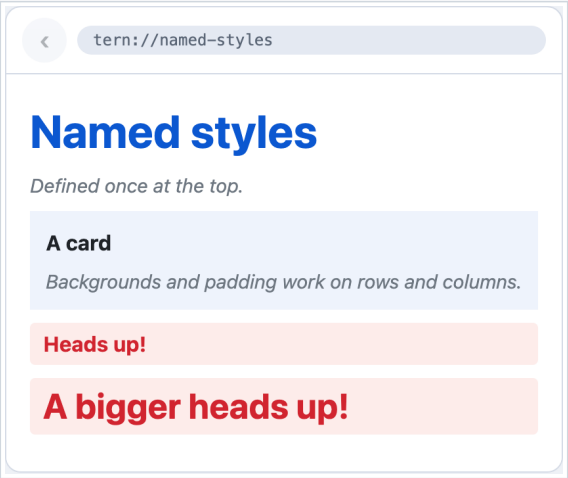
If you keep repeating the same settings, give them a name. A `style` goes at the top of the page, and then any element can use it with `style name`.

styles.tern

```
page "Named styles"

style title { size 26 bold color blue }
style note { italic color gray size 11 }
style warning { bold color red background #fdecea }
style card { background #eef3fc padding 12 }

view {
  text "Named styles" style title
  text "Defined once at the top." style note
  column style card {
    text "A card" bold
    text "Backgrounds and padding work on rows and columns." style note
  }
  text "Heads up!" style warning
  text "A bigger heads up!" style warning size 20
}
```



Four styles, used in five places

How styles combine

- List several names to use them together: `style title note`. Later ones win if they disagree.
- Settings written directly on the element win over a style, in the order you write them. `style warning size 20` is the warning style, but bigger.
- A style can hold any property. When you use it on an element, properties that element cannot have are simply ignored, so one style can serve text and cards alike.
- A style cannot use another style. Keep each one small and combine them on the element.

> **Tip.** Name styles by what they mean (`warning`, `note`, `card`), not by how they look (`red`). Then you can change the look in one place later.

12. Pages and navigation

Until now a page had one `view`. Give a view a name and you can have as many as you like. The first one is where the page starts. A button moves to another page with `goto`, and `back` returns to the page you came from.

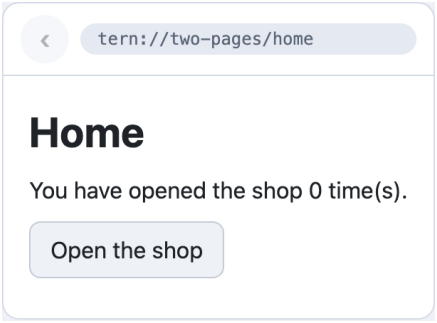
nav_basic.tern

```
page "Two pages"

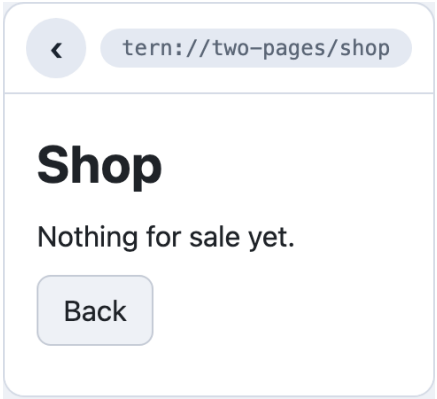
state visits: Int = 0

view home {
  heading "Home"
  text "You have opened the shop " + toText(visits) + " time(s)."
  button "Open the shop" -> {
    visits = visits + 1
    goto shop
  }
}

view shop {
  heading "Shop"
  text "Nothing for sale yet."
  button "Back" -> back
}
```



The home page



After tapping the button

- `goto shop` shows the page called `shop`. The name must match a `view shop { ... }`, and Tern checks it.
- `back` goes to the page you came from. On the first page it does nothing.
- A button can do several things in braces, such as changing state and then moving on.
- A view with no name, `view { ... }`, is called `home`.
- All pages share the same state, so what one page changes, the next page can show.

A small site

Pages mix with everything else in this guide: images, fonts, tables and styled buttons.

cafe.tern

```
page "Corner Cafe"

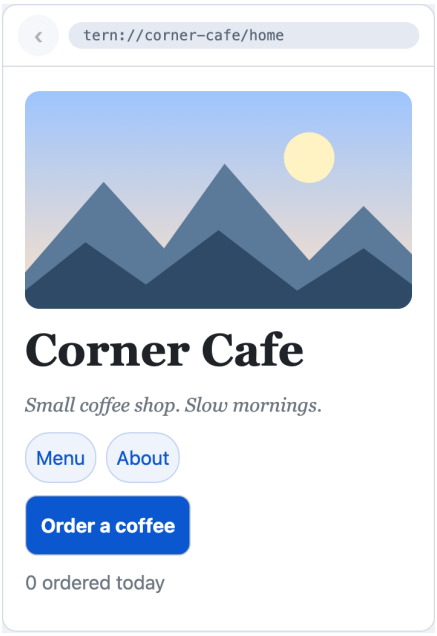
style chip { background #eef3fc color blue radius 20 padding 8 border #c5d3f0 }
style cta { background blue color white bold radius 10 padding 12 }

state orders: Int = 0
state names: List<Text> = ["Espresso", "Flat white", "Croissant"]
state prices: List<Float> = [2.5, 3.2, 4.0]

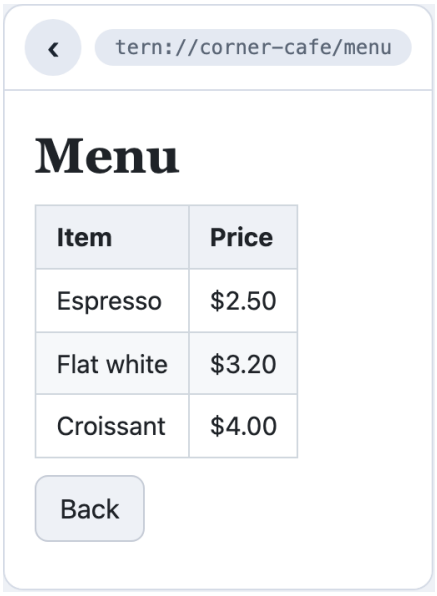
// Each "view name" is a page. Buttons and links move between them.
view home {
  image "mountains" width 320 radius 12 alt "Mountains at dawn"
  heading "Corner Cafe" font display size 28
  text "Small coffee shop. Slow mornings." font serif italic color gray
  row {
    button "Menu" style chip -> goto menu
    button "About" style chip -> goto about
  }
  button "Order a coffee" style cta -> orders = orders + 1
  text toText(orders) + " ordered today" color gray
}

view menu {
  heading "Menu" font display
  table {
    header "Item", "Price"
    for i in range(len(names)) {
      cells names[i], "$" + fixed(prices[i], 2)
    }
  }
  button "Back" -> back
}

view about {
  heading "About us" font display
  para {
    span "We opened in "
    span "2019" bold
    span " and we are open every day."
  }
  row {
    link "Home" -> goto home
    link "Our website" to "https://example.com"
  }
}
```



The home page



The menu page

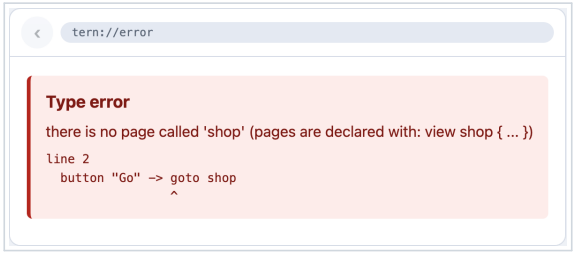
Links

A `link` can move between pages too: `link "Home" -> goto home`. To open a web address, write `link "Our website" to "https://example.com"`. In the playground, the round arrow beside the address does the same as `back`.

Typos are caught

A page that does not exist

```
view home {
  button "Go" -> goto shop
}
```



Tern says: there is no page called 'shop' (pages are declared with: view shop { ... })

13. Buttons and boxes in style

Yes, buttons can be styled. They take the same words as text, plus a few that make them look like buttons.

Add this	To get
`background blue`	the button colour
`color white`	the label colour
`size 16 bold italic`	label size and weight
`font serif`	the label font (chapter 14)
`padding 12`	space around the label, in pixels
`radius 10`	rounded corners. A big number such as 999 makes a pill
`border blue`	a thin outline
`width 280`	a fixed width in pixels

buttons.tern

```
page "Buttons"

style primary { background blue color white bold radius 10 padding 12 }
style ghost { color blue border blue radius 10 padding 10 background white }
style danger { background red color white radius 6 padding 10 }
style pill { background #e8f5e9 color green radius 999 padding 8 bold }

state clicks: Int = 0
state on: Bool = false

view {
  heading "Styled buttons"
  button "Primary" style primary -> clicks = clicks + 1
  button "Ghost" style ghost -> clicks = clicks + 1
  button "Danger (resets)" style danger -> clicks = 0
  button "Pill +5" style pill -> clicks = clicks + 5
  button "Wide button" style primary width 280 -> clicks = clicks + 1
  button "Serif italic" font serif italic size 16 -> clicks = clicks + 1
  button "Toggle" style ghost bold when on -> on = !on
  text "Clicks: " + toText(clicks) size 20 bold
  text "Toggle is " + toText(on) color gray
}
```



Four named styles, and a button that changes while you use it

Notice `bold when on` on the Toggle button: like text, a button's bold, italic and strike can depend on state. That is how you make a selected option look different.

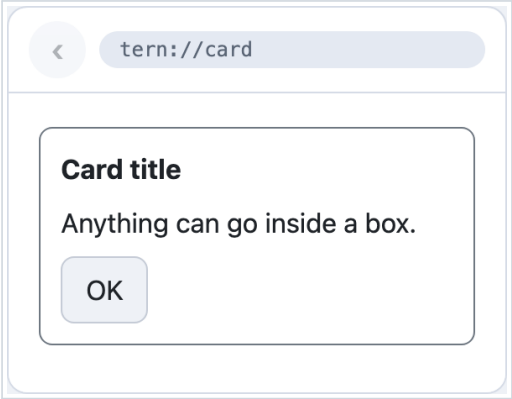
Boxes

A `row` or `column` can have a `background`, `padding`, `radius`, `border` and `width`, which turns it into a card.

box.tern

```
page "Card"

view {
  column border gray radius 8 padding 12 width 260 {
    text "Card title" bold
    text "Anything can go inside a box."
    button "OK" -> goto home
  }
}
```



A column styled as a card

Spacing, alignment and grids

A `row` or `column` takes `gap` (space between its items, default 8). Any block takes `margin` (space around it). On a `row`, `align` places the items: `left`, `center`, `right` or `spread`, which pushes them to opposite ends, like a menu bar. On a `column`, `align` places each item inside it left, centre or right. To put a narrow card in the middle of the page, wrap it in an outer `column align center`.

`grow N` makes a block in a row take N shares of the width that is left, so two columns can be one part and three parts. `maxwidth` and `minwidth` limit how wide a block gets. A `grid 3 { ... }` lays its items out in three equal columns, wrapping to new lines, with its own `gap`.

Layout

```
row align spread { text "Shop"  text "Cart" }

row gap 12 {
  column grow 1 { text "Side" }
  column grow 3 { text "Main" }
}

grid 3 gap 10 {
  for i in range(6) { text "Card " + toText(i) }
}

column align center {
  column maxwidth 360 padding 12 background #f6f8fa { text "Centered card" }
}
```

14. Fonts

Add `font` to any text, heading, bullet, paragraph, span, link or button. There are six to choose from.

Name	Feel	Good for
`sans`	clean, the default	body text
`serif`	bookish	articles and quotes
`mono`	fixed width	code and numbers
`display`	elegant headline face	big titles
`hand`	handwriting	notes and signatures
`rounded`	soft and friendly	buttons and labels

fonts.tern

```
page "Fonts"

state pick: Font = serif

view {
  heading "Fonts" font display
  text "sans: The quick brown fox" font sans
  text "serif: The quick brown fox" font serif
  text "mono: The quick brown fox" font mono
  text "display: The quick brown fox" font display size 18
  text "hand: The quick brown fox" font hand size 20
  text "rounded: The quick brown fox" font rounded
  divider
  text "Pick a font below" font pick size 22
  row {
    button "serif" -> pick = serif
    button "hand" -> pick = hand
    button "mono" -> pick = mono
  }
}
```



After tapping hand

`Font` is a type like `Color`, so a state can hold one (`state pick: Font = serif`) and a named style can include one. Fonts are fetched from the web the first time a page uses them. If a font cannot load, the browser shows a similar one instead.

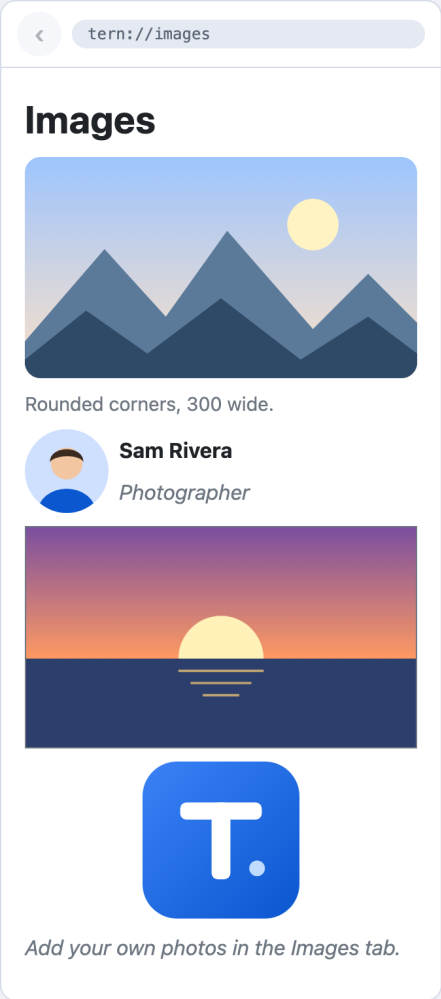
15. Images

An `image` shows a picture by name. Four sample pictures come with the playground (`mountains`, `sunset`, `avatar` and `logo`), and you can add your own.

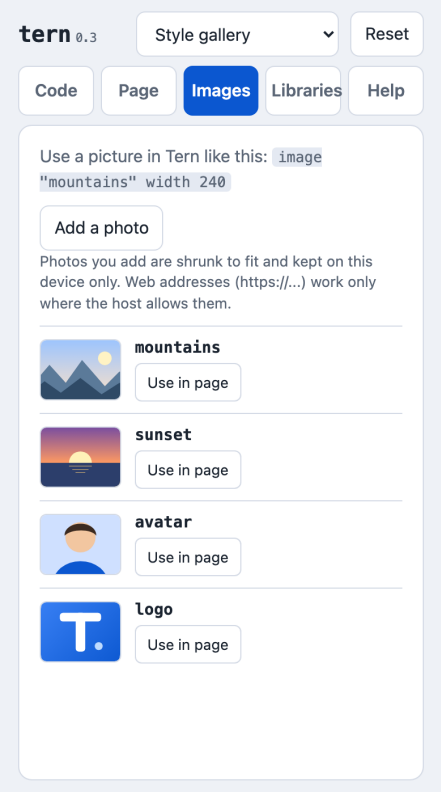
images.tern

```
page "Images"

view {
  heading "Images"
  image "mountains" width 300 radius 12 alt "Mountains at dawn"
  text "Rounded corners, 300 wide." color gray size 11
  row {
    image "avatar" width 64 height 64 radius 999 alt "A friendly face"
    column {
      text "Sam Rivera" bold
      text "Photographer" italic color gray
    }
  }
  image "sunset" width 300 border gray alt "Sunset over water"
  image "logo" width 120 align center alt "Tern logo"
  text "Add your own photos in the Images tab." italic color gray
}
```



The page



The Images tab

Add this	To get
`width 300 height 64`	a size in pixels. Give only the width and the height follows
`radius 12`	rounded corners. `radius 999` on a square image makes a circle
`border gray`	a thin outline
`align center`	`left`, `center` or `right`
`alt "text"`	a description, used if the picture cannot show

Adding your own photos

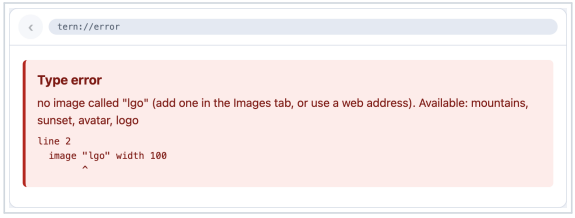
- Open the **Images** tab and tap **Add a photo**. Pick one from your phone.
- The photo is shrunk to a sensible size and named after its file, such as `team-photo`. It stays on this device only.
- Tap **Use in page** to drop `image "team-photo" width 240` into your code.
- **Remove** takes it away again.

Name checking

Tern checks the name of every picture you write in quotes, and lists the ones you have.

A misspelled picture name

```
view {
  image "lgo" width 100
}
```



Tern says: no image called "lgo" (add one in the Images tab, or use a web address). Available: mountains, sunset, avatar, logo

A picture name can also come from state, as in `state pic: Text = "logo"` and `image pic`, which is a good way to let a button change the picture.

16. Decimal numbers

`Float` holds numbers with decimals, written with a point: `2.5`, `0.0`, `100.0`. A `Float` is a different type from an `Int`, so Tern never mixes them by accident.

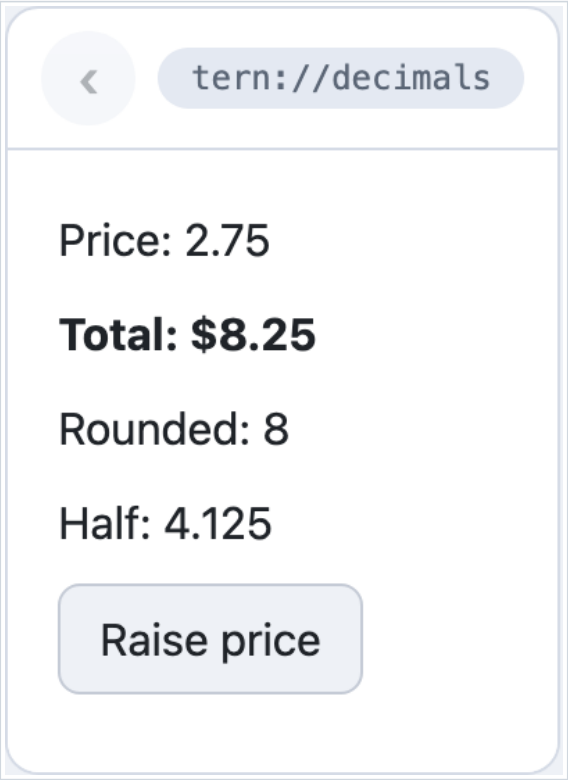
float.tern

```
page "Decimals"

state price: Float = 2.5
state qty: Int = 3

fn total(): Float {
  return price * toFloat(qty)
}

view {
  text "Price: " + toText(price)
  text "Total: $" + fixed(total(), 2) bold
  text "Rounded: " + toText(round(total()))
  text "Half: " + toText(total() / 2.0)
  button "Raise price" -> price = price + 0.25
}
```



After tapping Raise price once

To go from	Write
Int to Float	`toFloat(qty)`
Float to Int	`round(x)` `floor(x)` `ceil(x)`
Float to Text, tidy	`fixed(x, 2)`, which gives `"7.50"`
Float to Text, as is	`toText(x)`, which gives `"7.5"`
typed Text to Float	`toFloatOr(text, 0.0)`, which uses the fallback if the text is not a number
typed Text to Int	`toIntOr(text, 0)`

Division with two `Float` values is exact (`7.0 / 2.0` is `3.5`). Division with two `Int` values still rounds down. `%` works only on `Int`.

Mixing Int and Float

```
state n: Int = 2

view {
  text toText(n * 1.5)
}
```



Tern says: '*' cannot mix Int and Float (use toFloat(...) to turn an Int into a Float)

Example: a tip calculator

tipcalc.tern

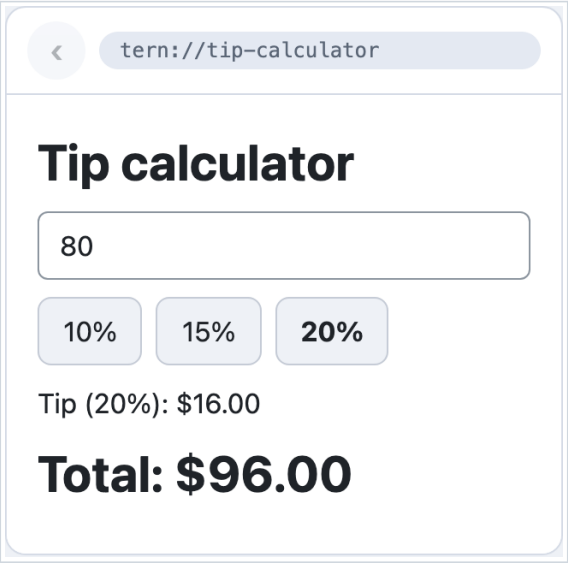
```
page "Tip calculator"

state bill: Text = ""
state percent: Int = 15

// Int and Float are different types. toFloat() converts, and
// toFloatOr() turns typed text into a number (or uses the fallback).
fn amount(): Float {
  return toFloatOr(bill, 0.0)
}

fn tip(): Float {
  return amount() * toFloat(percent) / 100.0
}

view {
  heading "Tip calculator"
  input bill placeholder "Bill amount, e.g. 42.50"
  row {
    button "10%" bold when percent == 10 -> percent = 10
    button "15%" bold when percent == 15 -> percent = 15
    button "20%" bold when percent == 20 -> percent = 20
  }
  text "Tip (" + toText(percent) + "%): $" + fixed(tip(), 2)
  text "Total: $" + fixed(amount() + tip(), 2) size 22 bold
}
```



After typing 80 and tapping 20%

The text box gives `Text`, so `toFloatOr` turns it into a number, and if someone types nonsense the calculator shows zero instead of breaking.

17. Structs: your own types

So far each fact lived in its own state. A shop that needs names, prices and quantities would need three lists that you must keep in step. A `struct` groups the facts about one thing into a single value with a name of its own, and can carry the small functions that work on it, called **methods**.

struct_basic.tern

```
page "Profile"

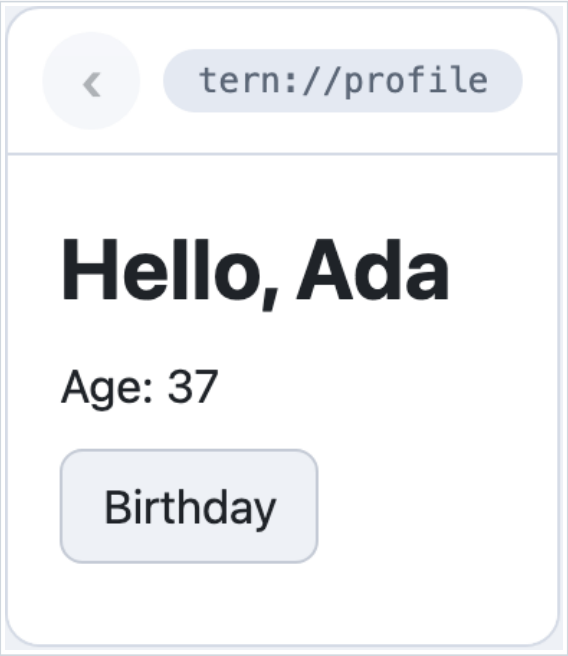
struct Person {
  name: Text
  age: Int

  fn greeting(): Text {
    return "Hello, " + self.name
  }

  fn birthday() {
    self.age = self.age + 1
  }
}

state me: Person = Person(name: "Ada", age: 36)

view {
  heading me.greeting()
  text "Age: " + toText(me.age)
  button "Birthday" -> me.birthday()
}
```



After tapping Birthday once

To	Write
declare a struct (at the top, outside `view`)	`struct Person { name: Text age: Int }`, one field per line
make one	`Person(name: "Ada", age: 36)`. Every field, by name
read a field	`me.name` `cart[0].price`
change a field	`me.age = 37` `cart[0].qty = cart[0].qty + 1`
make a changed copy	`me with age: 37`, or several: `me with name: "Grace", age: 40`
use it as a type	`state me: Person`, `fn birthday(p: Person): Person`, `List<Person>`

Methods

A `fn` inside a struct is a method. It reaches the struct it was called on through `self`. Call it with a dot: `me.greeting()`.

- A method that **returns a value**, like `greeting()`, only reads `self`.
- A method that **returns nothing**, like `birthday()`, may change `self`. Call it as its own statement, such as a button's action, on a variable or on a list item: `me.birthday()`, `cart[0].more()`.
- A method sees only `self` and its own arguments, so a struct in a library works the same as one in a page.

Structs are values

A struct is a value, like a number. `let b = a` makes a copy, and changing `b` never changes `a`. Under the hood a change such as `me.birthday()` builds a new `Person` and stores it back in `me`. That keeps every change visible in your page's state and makes surprises impossible: two names can never secretly share one struct.

The same idea explains the one rule that trips people up. A loop variable is a copy of a list item, so a method that changes it would be lost. Tern refuses, and tells you to change the list instead.

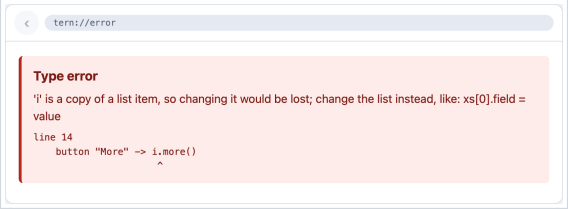
Changing a copy

```
struct Item {
  name: Text
  qty: Int

  fn more() {
    self.qty = self.qty + 1
  }
}

state cart: List<Item> = [Item(name: "Tea", qty: 1)]

view {
  for i in cart {
    button "More" -> i.more()
  }
}
```



Tern says: 'i' is a copy of a list item, so changing it would be lost; change the list instead, like: xs[0].field = value

Lists of structs

Structs shine in lists: one list, and every row is a whole item. `cart[0].more()` reaches into the list, changes that one item, and stores the list back.

struct_cart.tern

```
page "Cart"

struct Item {
  name: Text
  price: Int
  qty: Int

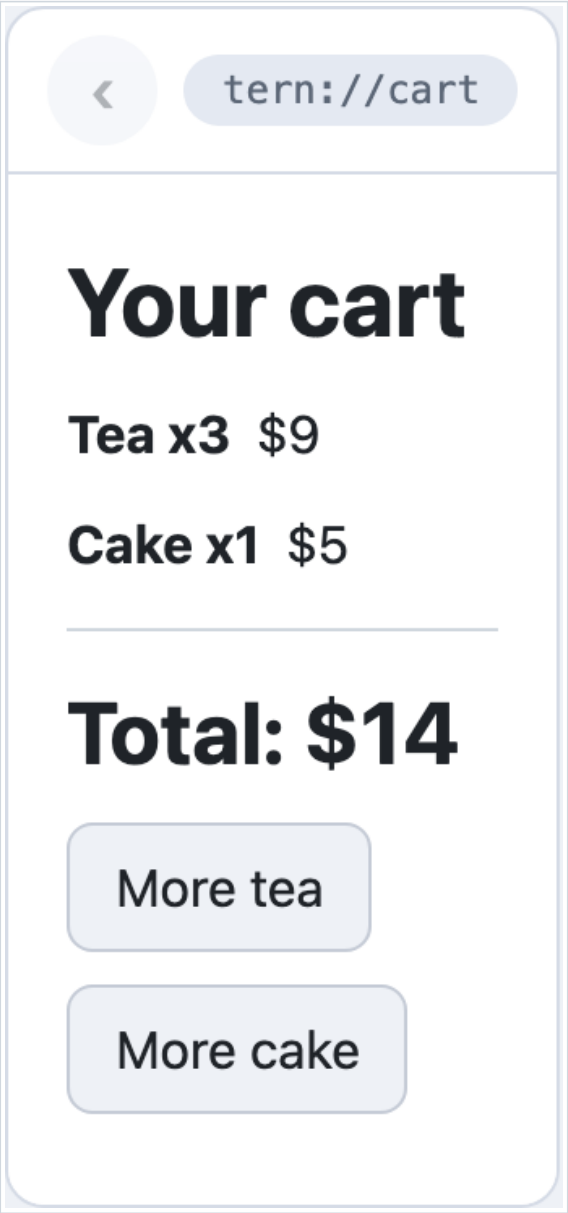
  fn lineTotal(): Int {
    return self.price * self.qty
  }

  fn more() {
    self.qty = self.qty + 1
  }
}

state cart: List<Item> = [
  Item(name: "Tea", price: 3, qty: 2),
  Item(name: "Cake", price: 5, qty: 1)
]

fn total(): Int {
  let sum = 0
  for i in cart { sum = sum + i.lineTotal() }
  return sum
}

view {
  heading "Your cart"
  for i in cart {
    row {
      text i.name + " x" + toText(i.qty) bold
      text "$" + toText(i.lineTotal())
    }
  }
  divider
  text "Total: $" + toText(total()) size 20 bold
  button "More tea" -> cart[0].more()
  button "More cake" -> cart[1].more()
}
```



After tapping More tea

Named fields keep you safe

You must give every field, by name, so a missing or misspelled field is caught before the page runs. Giving values by position, as in `Item("Tea", 3)`, is not allowed.

A missing field

```
struct Item {
  name: Text
  price: Int
}

view {
  text Item(name: "Tea").name
}
```



Tern says: Item needs price: Int

Position-only values get a clear message too: **syntax error at line 7, column 13: Item(...) needs named fields, like Item(field: value, ...)**

> **Note. Rules of structs.** Two different structs are different types, even with the same fields. `==` compares two values of the same struct field by field. A struct can hold other structs and lists, but never itself. Field names cannot be reserved words, so `size` and `color` are out (try `boxSize`). A method that returns a value cannot change `self`: Tern says **method 'next' returns a value, so it cannot change self; return a changed copy with 'with', or make it return nothing**. `toText` does not take a struct; use it on a field, as in `toText(me.age)`. To change one item of a list you may write `cart[0].qty = 3`, but a whole item is still replaced with `setAt`.

18. Libraries: sharing logic

As pages grow, the rules (how a total is worked out, how money is shown) get mixed up with the layout, and a second page needs the same rules again. A **library** is a separate file that holds only the rules. It ends in `.ternlib`.

shop.ternlib

```
// shop.ternlib: shared logic. No drawing, no state.
struct Item {
  name: Text
  price: Int
  qty: Int
}

fn make(name: Text, price: Int): Item {
  return Item(name: name, price: price, qty: 1)
}

fn lineTotal(i: Item): Int {
  return i.price * i.qty
}

fn total(items: List<Item>): Int {
  let sum = 0
  for i in items { sum = sum + lineTotal(i) }
  return sum
}

fn money(n: Int): Text {
  return "$" + toText(n)
}
```

A page uses it with `use`, near the top. Everything that comes from the library is written with the library's name in front, so it can never clash with your own names.

shop.tern

```
page "Shop"

use "shop.ternlib"

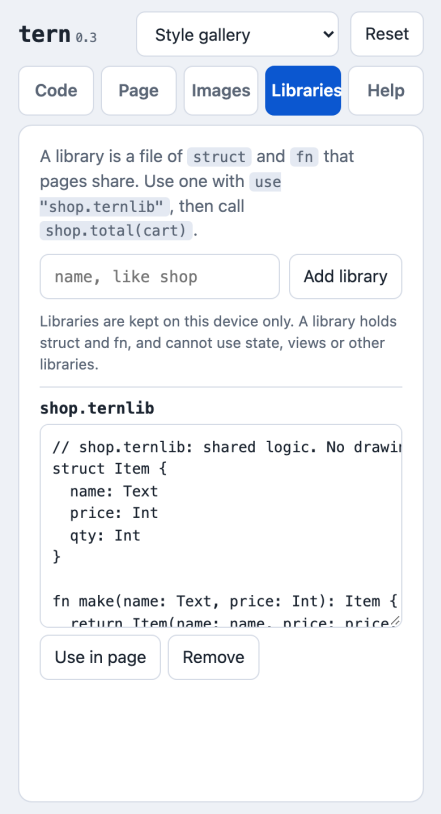
state cart: List<shop.Item> = [shop.make("Tea", 3), shop.make("Cake", 5)]

fn addTea() {
  cart = append(cart, shop.make("Tea", 3))
}

view {
  heading "Your cart"
  for i in cart {
    row {
      text i.name
      text shop.money(shop.lineTotal(i))
    }
  }
  divider
  text "Total: " + shop.money(shop.total(cart)) bold size 20
  button "Add tea" -> addTea()
}
```



The page, after Add tea



The Libraries tab

Write	To
`use "shop.ternlib"`	load the library file `shop.ternlib`
`shop.total(cart)`	call the library's function `total`
`shop.Item`	use its struct as a type, as in `List<shop.Item>`
`shop.Item(name: "Tea", price: 3, qty: 1)`	make one of its structs

Where the library lives

- **In the playground**, open the **Libraries** tab, type a name such as `shop`, tap **Add library** and write the code. **Use in page** adds the `use` line for you. Libraries stay on this device.
- **In Tern Browser**, put `shop.ternlib` in the same folder as the page. A page can only name a file that sits beside it: no folders, no addresses.

What a library may hold

Only `struct` and `fn`. That is the point: a library can work things out, but it cannot draw anything, keep `state`, change pages with `goto`, or load other libraries. A library cannot read the page's state either; give the function what it needs as arguments. Because of this, a library from someone else can never change what your page does behind your back. Structs in a library can have methods too.

A second page can reuse exactly the same rules:

receipt.tern

```
page "Receipt"

use "shop.ternlib"

state order: List<shop.Item> = [
  shop.Item(name: "Soup", price: 6, qty: 2),
  shop.make("Bread", 2)
]

view {
  heading "Receipt"
  for i in order {
    text i.name + ": " + shop.money(shop.lineTotal(i))
  }
  text "Pay " + shop.money(shop.total(order)) bold
}
```

Errors

A page that asks for something the library does not have is caught before it runs, and Tern lists what the library does have.

A misspelled library function

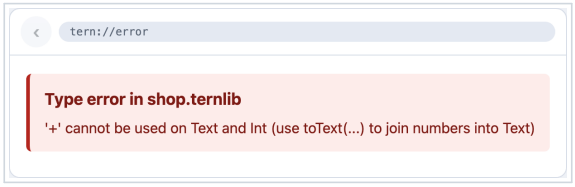
```
use "shop.ternlib"

view {
  text shop.totl(5)
}
```



Tern says: library shop has no 'totl' (it has: make, lineTotal, total, money, Item)

A mistake inside the library is reported with the library's file name and its own line numbers. The error card shows the library's code.



Tern says: type error in shop.ternlib at line 23, column 14: '+' cannot be used on Text and Int (use toText(...) to join numbers into Text)

> **Note. Good to know.** Library names are letters, digits and _, followed by `.ternlib`, and cannot be a reserved word. Do not use a library's name for a state, function, parameter or variable. Libraries cannot yet use other libraries, and pages opened from a web address cannot load libraries yet.

19. Reading error messages

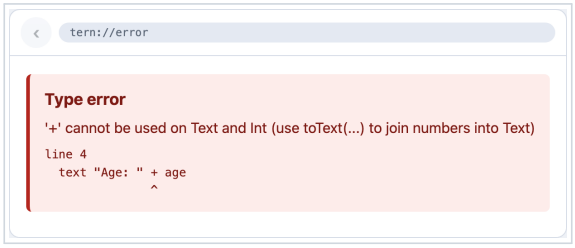
Tern reports three kinds of problem. Every message gives a line and column, and the Page tab also shows the line of code. Read the message literally; it usually tells you the fix.

Type errors: caught before the page runs

Mixing Text and Int (e_types.tern)

```
state age: Int = 30

view {
  text "Age: " + age
}
```



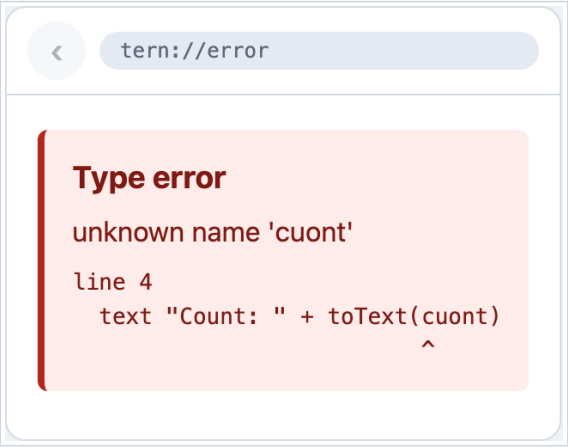
Tern says: '+' cannot be used on Text and Int (use toText(...) to join numbers into Text)

Convert with `toText(age)`.

A typo in a name (e_name.tern)

```
state count: Int = 0

view {
  text "Count: " + toText(cuont)
}
```



Tern says: unknown name 'cuont'

`cuont` is not defined. Check spelling.

A function that forgets to return (e_ret.tern)

```
fn half(n: Int): Int {
  if n > 0 {
    return n / 2
  }
}

view {
  text toText(half(4))
}
```



Tern says: function 'half' must return Int on every path

If `n` is not greater than 0, nothing is returned.

Syntax errors: the file is not valid Tern

A missing closing brace

```
view {
  heading "Oops"
  text "No closing brace"
```

Tern says: **syntax error at line 4, column 1: missing '}'**

Runtime errors: the page works until one specific action goes wrong

Dividing by zero

```
state zero: Int = 0
state result: Int = 0

view {
  button "Divide" -> result = 10 / zero
  text toText(result)
}
```

Clicking Divide shows this above the page: **runtime error at line 5, column 34: division by zero**

The page keeps working, and only that action is cancelled. Runtime errors come from dividing by zero, indexing outside a list, and recursing too deeply.

20. Project: a quiz

Time to combine everything. This page asks questions one at a time, checks answers, keeps score, and offers a replay. It uses state, two functions, `input`, `button`, `if`/`else` in the view, lists, indexing, and text built from pieces.

quiz.tern

```
page "Quiz"

state questions: List<Text> = ["What is 2 + 2?", "Capital of France?", "Is 7 prime?"]
state answers: List<Text> = ["4", "Paris", "yes"]
state index: Int = 0
state score: Int = 0
state guess: Text = ""
state feedback: Text = ""

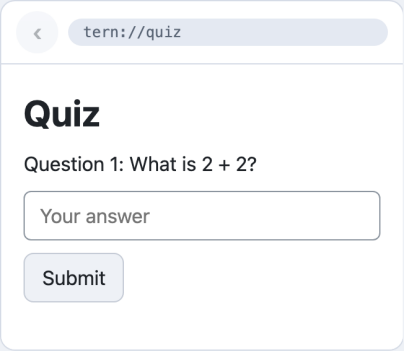
fn submit() {
  if lower(trim(guess)) == lower(answers[index]) {
    score = score + 1
    feedback = "Correct!"
  } else {
    feedback = "Not quite. The answer was " + answers[index] + "."
  }
  guess = ""
  index = index + 1
}

fn restart() {
  index = 0
  score = 0
  feedback = ""
}

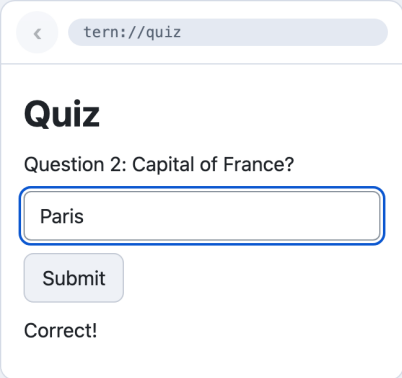
view {
  heading "Quiz"
  if index < len(questions) {
    text "Question " + toText(index + 1) + ": " + questions[index]
    input guess placeholder "Your answer"
    button "Submit" -> submit()
  } else {
    text "Finished! Score: " + toText(score) + " out of " + toText(len(questions))
    button "Play again" -> restart()
  }
  text feedback
}
```

How it works

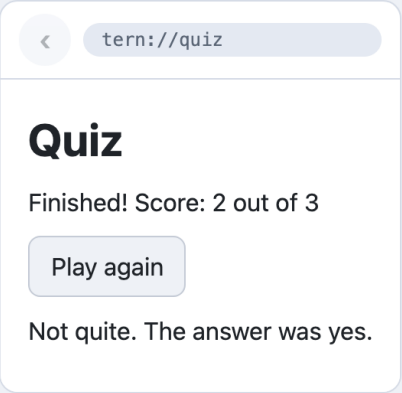
- `index` says which question we are on. `questions[index]` fetches its text.
- While `index < len(questions)` the view shows the question, a box and a Submit button. Once we run out, the `else` branch shows the final score.
- `submit()` compares the guess to the answer. `lower` and `trim` ignore capital letters and stray spaces. Then it clears the box and moves on.
- `feedback` holds the message after each answer, and the view just displays it.



Start



Mid-game



The end

Ideas to extend it

- Add more questions, remembering to add an answer for each.
- Show the running score while playing.
- Add a `link` to a second quiz page on another topic.

What Tern 0.3 does not have (yet)

User-defined record types, saving data between visits, animation, and pages that fetch data from the network. Pages cannot read files or reach the internet, which keeps them safe to open, but it also means this is a learning language, not yet a replacement for a full web stack.

21. Exercises

Try each one before reading the solutions on the next pages. The status line under your code tells you straight away whether the page type-checks.

1. Full name

Make a page with two text boxes, first name and last name, that shows the full name underneath. (Hint: two `Text` states and `+`.)

2. A counter with a floor

Take the counter from chapter 4 and make the minus button stop at zero.

3. Clear all

Make a page that lists three items and has a button that removes them all at once.

4. FizzBuzz

Show the numbers 1 to 15, one per line. Print `Fizz` instead of multiples of 3, `Buzz` instead of multiples of 5, and `FizzBuzz` for multiples of both. (Hint: write a function `fizz(n: Int): Text`, and use `%`.)

5. A profile card

Make a card: a column with a light blue background and some padding, holding a heading, a gray italic job title, and a paragraph in which one phrase is bold. (Hint: a named style for the card.)

6. Expenses

Show three expenses in a table with a final Total row. Add the numbers up in a function.

7. A two-page site

Make a home page with a heading and a button that opens a contact page. The contact page shows an email address and a Back button. Give both buttons a named style.

8. Split the bill

A text box takes the bill. Plus and minus buttons change the number of people (never below one). Show what each person pays with two decimals. (Hint: `toFloatOr`, `toFloat`, `fixed`.)

9. Break it on purpose

Take any working page and introduce one mistake: a wrong type, a misspelled name, a missing `return`. Predict the error message before you type it in.

Solutions

1. Full name

```
page "Full name"

state first: Text = ""
state last: Text = ""

view {
  input first placeholder "First name"
  input last placeholder "Last name"
  text "Full name: " + trim(first + " " + last)
}
```

`trim` tidies the result when one of the boxes is empty.

2. A counter with a floor

```
page "Counter with a floor"

state count: Int = 0

fn down() {
  if count > 0 {
    count = count - 1
  }
}

view {
  text "Count: " + toText(count)
  row {
    button "-" -> down()
    button "+" -> count = count + 1
  }
}
```

The `if` inside `down()` only subtracts when the count is above zero.

3. Clear all

```
page "Todo with clear"

state items: List<Text> = ["One", "Two", "Three"]

view {
  for item in items {
    text item
  }
  button "Clear all" -> items = []
}
```

`[]` is allowed here because Tern already knows `items` is a `List<Text>`.

4. FizzBuzz

```
page "FizzBuzz"

fn fizz(n: Int): Text {
  if n % 15 == 0 { return "FizzBuzz" }
  if n % 3 == 0 { return "Fizz" }
  if n % 5 == 0 { return "Buzz" }
  return toText(n)
}

view {
  for i in range(15) {
    text fizz(i + 1)
  }
}
```

Check the multiples of 15 first, otherwise 15 would print just `Fizz`.

5. A profile card

```
page "Profile card"

style card { background #eef3fc padding 14 }

view {
  column style card {
    heading "Ada Lovelace" size 20
    text "Mathematician" italic color gray
    para {
      span "Wrote the "
      span "first program" bold
      span " in 1843."
    }
  }
}
```

`column style card` applies the style's background and padding. The `span` in bold stands out inside the paragraph.

6. Expenses

```
page "Expenses"

state labels: List<Text> = ["Rent", "Food", "Travel"]
state costs: List<Int> = [900, 300, 120]

fn total(): Int {
  let sum = 0
  for c in costs {
    sum = sum + c
  }
  return sum
}

view {
  table {
    header "Item", "Cost"
    for i in range(len(labels)) {
      cells labels[i], "$" + toText(costs[i])
    }
    cells "Total", "$" + toText(total())
  }
}
```

`cells` can sit after the `for`. The last row adds the Total by calling `total()`.

7. A two-page site

```
page "My site"

style btn { background blue color white radius 8 padding 10 }

view home {
  heading "Welcome"
  button "Contact me" style btn -> goto contact
}

view contact {
  heading "Contact"
  text "hello@example.com"
  button "Back" style btn -> back
}
```

The first view is the start page. `goto contact` moves forward and `back` returns.

8. Split the bill

```
page "Split the bill"

state bill: Text = ""
state people: Int = 2

fn fewer() {
  if people > 1 {
    people = people - 1
  }
}

fn each(): Float {
  return toFloatOr(bill, 0.0) / toFloat(people)
}

view {
  input bill placeholder "Bill"
  row {
    button "-" -> fewer()
    text toText(people) + " people"
    button "+" -> people = people + 1
  }
  text "Each pays $" + fixed(each(), 2) bold
}
```

`people` is an `Int` and the bill is a `Float`, so `toFloat(people)` converts before dividing. `fewer()` keeps the count at one or more.

Appendix A: Quick reference

Reserved words

`page state fn view let if else for foreach in return true false to placeholder text heading button input row column link divider style size color background align padding bold italic underline strike when para span table header cells bullet image alt width height radius border font goto back struct with use self checkbox resizable modal gap margin grow maxwidth minwidth grid select radio textarea slider disabled saved every`. You cannot use these as names. Type names (`Int`, `Text`, `Bool`, `List`) start with a capital letter. The built-in colour and alignment names (`black white gray red green blue orange purple yellow pink`, `left center right`, `sans serif mono display hand rounded`) can be used but not changed.

Built-in functions

Function	Type	What it does
`toText(x)`	Int, Bool or Text -> Text	Convert to text
`len(x)`	Text or List -> Int	Length
`append(xs, x)`	List<T>, T -> List<T>	New list with x on the end
`removeAt(xs, i)`	List<T>, Int -> List<T>	New list without item i
`range(n)`	Int -> List<Int>	0 up to n-1 (at most 10000)
`slice(xs, a, b)`	List<T>, Int, Int -> List<T>	Items a up to b-1; out-of-range is trimmed
`sort(xs)`	List<Int|Float|Text> -> same	New list, smallest first
`sortOrder(xs)`	List<Int|Float|Text> -> List<Int>	Positions in sorted order
`reverse(xs)`	List<T> -> List<T>	New list, back to front
`trim(s)`	Text -> Text	Remove spaces at both ends
`upper(s)` / `lower(s)`	Text -> Text	Change case
`contains(s, part)`	Text, Text -> Bool	Is part inside s?
`join(xs, sep)`	List<Text>, Text -> Text	Glue items together
`setAt(xs, i, x)`	List<T>, Int, T -> List<T>	New list with item i replaced
`rgb(r, g, b)`	Int, Int, Int -> Color	A colour; each part 0 to 255
`toFloat(n)`	Int -> Float	Whole number to decimal
`round(x)` `floor(x)` `ceil(x)`	Float -> Int	Nearest, down, up
`fixed(x, places)`	Float, Int -> Text	Text with a set number of decimals
`sqrt(x)`	Float -> Float	Square root
`toIntOr(s, fallback)`	Text, Int -> Int	Typed text to a number, or the fallback
`toFloatOr(s, fallback)`	Text, Float -> Float	Typed text to a decimal, or the fallback

Structs and libraries

Write	Meaning
`struct Item { name: Text price: Int }`	declare a struct, one field per line
`Item(name: "Tea", price: 3)`	make one: every field, by name
`item.name`	read a field
`item.price = 4`	change a field (on a variable or a list item of one)
`item with price: 4`	a changed copy; several fields with commas
`fn more() { self.qty = self.qty + 1 }`	a method inside a struct; no return value, so it may change `self`
`item.more()` `cart[0].more()`	call a method with a dot
`use "shop.ternlib"`	load a library (`struct` and `fn` only)
`shop.total(xs)` `shop.Item`	a library's function or struct, always with its name in front

Style properties

Property	Value	Works on
`size 20`	Int, points (6 to 96)	text, heading, bullet, para, span, link, button
`bold` `italic` `strike`	optional `when Bool`	text, heading, bullet, para, span, link, button
`underline`	optional `when Bool`	text, heading, bullet, para, span
`color red`	Color	text, heading, bullet, para, span, link, button
`background #fff3bf`	Color	text, heading, bullet, para, span, button, row, column
`font serif`	Font	text, heading, bullet, para, span, link, button
`align center`	Align	text, heading, bullet, para; image
`padding 12`	Int, pixels (0 to 64)	text, heading, bullet, para, button, row, column
`radius 10`	Int, pixels (0 to 999)	text, heading, bullet, para, button, row, column, image
`border gray`	Color	text, heading, bullet, para, button, row, column, image
`width 200`	Int, pixels	button, row, column, image
`height 100`	Int, pixels	image
`alt "text"`	Text	image
`style a b`	named styles, in order	anything above (unused properties are ignored)

Operator order (tightest first)

function call and `xs[i]`, then unary `!` and `-`, then `* / %`, then `+ -`, then `< > <= >=`, then `== !=`, then `&&`, then `||`. Use parentheses when in doubt.

Appendix B: Common mistakes

You wrote	Fix
`text "Total: " + total`	`text "Total: " + toText(total)`
`if count { ... }`	`if count > 0 { ... }`
`items = append(items, x)` missing, only `append(items, x)`	Lists never change in place. Assign the result back.
`state xs: List<Int> = []` written as `let xs = []`	Give it a type: `let xs: List<Int> = []`
`input count` where `count` is an `Int`	Inputs only edit `Text` state
`button "Go" -> 1 + 2`	A button needs a call or an assignment
`return` in a button	Only functions can return; call a function instead
`text "x" color "red"`	Colours are not text. Write `color red`, with no quotes
`cells "Tea"` under a two-column header	Every row needs the same number of cells
`para { text "x" }`	Inside a paragraph, write `span "x"`
`toText(n * 1.5)` where `n` is an `Int`	Int and Float never mix. Use `toFloat(n) * 1.5`
`button "Go" -> goto shop` with no `view shop`	Declare the page: `view shop { ... }`
`image "lgo"`	Image names must match one in the Images tab
`Item("Tea", 3)`	Give every field by name: `Item(name: "Tea", price: 3)`
`for i in cart { i.more() }`	`i` is a copy, so the change is lost. Write `cart[0].more()`
a method that returns a value and sets `self.x`	Split it: one method changes, another returns
`struct Box { size: Int }`	`size` is a reserved word; try `boxSize`
`total(cart)` for a library function	Write the library name first: `shop.total(cart)`